

Performance Analysis

Intelligent Video Analytics Platform

1. Methodology

Two configurations were run end-to-end using `performance_analysis.py` against an identical 300-frame test video, on an NVIDIA Quadro T1000 (12 logical CPU cores). CPU and memory were sampled from a background thread on a fixed 0.25s interval (not per-frame, which gives unreliable `psutil` readings). The first 3 frames were excluded from timing statistics — the very first inference call includes one-time CUDA/model initialization cost (measured at 3.4-5.9 seconds here) that would otherwise distort every average.

Run	Model	Precision	Tracker	Confidence	Inference Size
A	YOLOv8n (nano)	fp32	ByteTrack	0.3	640
B	YOLOv8s (small)	fp16	ByteTrack	0.3	640

2. FPS

Metric	Run A (nano, fp32)	Run B (small, fp16)
Wall-clock FPS (frames ÷ total elapsed time)	26.66	13.21
Inference-only FPS (1000 ÷ steady-state ms)	68.41	16.44

These two numbers disagree substantially, and that gap is itself an important finding — see Section 7. In short: wall-clock FPS is the number that reflects what a user actually experiences (frame capture + inference + tracking + everything else in the loop); inference-only FPS reflects just the `model.track()` call in isolation. Wall-clock FPS is the correct number to quote for real-world performance.

3. Inference Time

Metric	Run A (nano, fp32)	Run B (small, fp16)
First frame (warmup, excluded from stats)	5893.45 ms	3398.63 ms
Steady-state average	14.62 ms	60.82 ms
Steady-state P95	23.58 ms	71.91 ms
Steady-state max	33.05 ms	75.84 ms

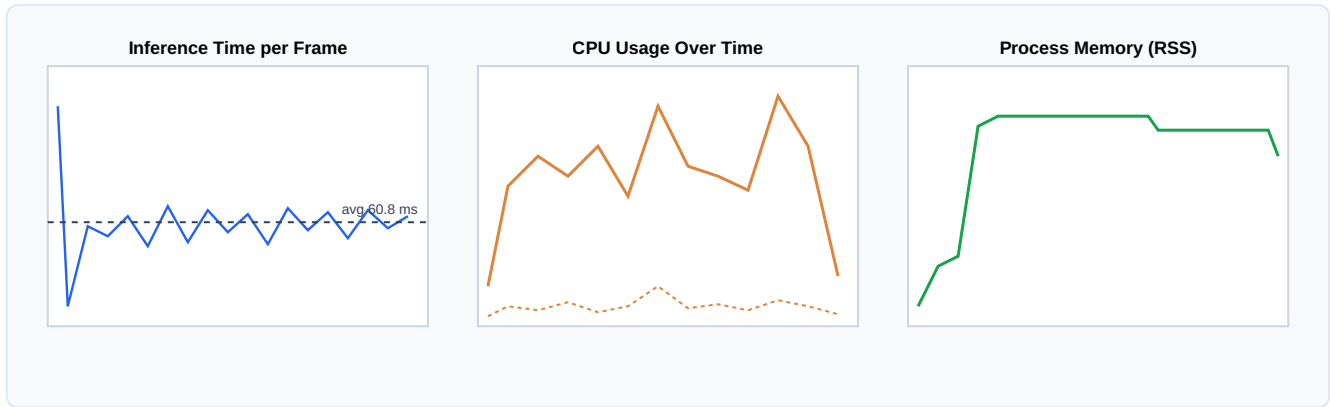


Figure 1 — Run B (YOLOv8s, fp16): inference time, CPU usage, and memory over the session. Warmup frame correctly excluded from the steady-state average and clipped from the y-axis for readability.

YOLOv8s at fp16 was roughly 4× slower per frame than YOLOv8n at fp32 (60.82ms vs 14.62ms) — model size dominates the precision-mode benefit here, consistent with the same finding in Assignment 2. fp16’s speedup exists but is a secondary optimization on top of model choice, not a substitute for it.

4. CPU Usage

Metric	Run A (nano, fp32)	Run B (small, fp16)
Process average	105.3%	102.86%
Process peak	206.2%	187.5%
System-wide average	17.48%	13.49%
Logical cores available	12	12

Process CPU >100% simply means more than one core was in use at once (e.g. 200% ≈ 2 cores) — normal `psutil` behavior, not a bug. Both runs used roughly one full core’s worth of CPU on average, with brief 2-core spikes, out of 12 available — CPU was not the bottleneck for either configuration, since the GPU was doing the actual inference work; this CPU usage is mostly frame decoding, Python overhead, and the tracker’s own (CPU-side) association logic.

5. Memory Usage

Metric	Run A (nano, fp32)	Run B (small, fp16)
Start	512.41 MB	548.54 MB
Average	907.59 MB	1190.38 MB
Peak	1120.01 MB	1270.17 MB
End	1063.67 MB	1172.17 MB

Memory climbs sharply in the first ~1-3 seconds (model + CUDA context loading) then plateaus for the rest of the session in both runs (visible in Figure 1’s memory panel) — there is no continuous upward creep over the 300-frame run, which is the important thing to check for: it means there’s no obvious memory leak in this pipeline at this

timescale. YOLOv8s used consistently more memory than YOLOv8n (peak 1270MB vs 1120MB), as expected from its larger parameter count.

6. Tracking Accuracy

True MOT accuracy metrics (MOTA, IDF1) require labeled ground-truth video with a correct ID assigned to every object in every frame — not available for this test clip. The proxies below are ground-truth-free but real: ID switch rate (how often a track's identity appears to restart) and average track length (how many frames a typical ID persists before being lost or replaced).

Metric	Run A (nano, fp32)	Run B (small, fp16)
Unique track IDs created	14	18
Average track length	62.57 frames	50.06 frames
ID switch events	42	57
ID switch rate (per 100 frames)	14.0	19.0

This result is counter-intuitive and worth discussing rather than glossing over: YOLOv8s — the larger, theoretically more accurate detector — showed a worse tracking-stability proxy than YOLOv8n on this same clip (more unique IDs, shorter average track length, higher switch rate). Two explanations are plausible and cannot be fully distinguished without ground truth: (1) YOLOv8s may be detecting additional smaller/harder objects that YOLOv8n misses entirely — new detections of previously-invisible objects would show up as "extra" short tracks in this proxy, which looks like instability but is arguably more thorough detection; or (2) YOLOv8s's detections may genuinely be less stable frame-to-frame for this content, causing real fragmentation. Distinguishing these two would require manually reviewing the annotated output video or obtaining labeled ground truth — this is a real limitation of proxy metrics, not a flaw to hide.

7. Key Finding — Why Wall-Clock and Inference-Only FPS Disagree

The FPS gap flagged in Section 2 is large enough to be worth understanding, not just reporting. Working backward from the numbers:

Run	Wall-clock time/ frame	Steady-state inference time/frame	Implied overhead/ frame	Inference share of total time
Run A (nano, fp32)	37.50 ms	14.62 ms	22.88 ms	39.0%
Run B (small, fp16)	75.70 ms	60.82 ms	14.88 ms	80.3%

The steady-state inference time only measures the `model.track()` call itself. Wall-clock FPS additionally includes everything else in the per-frame loop — reading the frame (`cv2.VideoCapture.read()`), Python-level bookkeeping, and any tracker association work that falls outside the timed window. That overhead turns out to be roughly 15-23ms per frame in both runs, similar in absolute terms — but because it is a fairly fixed cost, it consumes a much larger fraction of the budget when the model itself is fast (61% of every frame's time for YOLOv8n) than when the model is already slow (20% for YOLOv8s).

Practical implication: For a fast model like YOLOv8n, further speeding up the model alone (e.g. more aggressive fp16, smaller imgsz) gives diminishing real-world returns, because the majority of each frame's wall-clock time is no longer the model at all — the frame-capture/pipeline overhead becomes the actual bottleneck. Any future optimization work on this platform should profile and target that overhead specifically once a fast model is already in use, rather than continuing to optimize inference alone.

Separately, 11 of Run A's 300 frames (3.7%) recorded an implausible exact 0.0ms inference time — physically impossible for real GPU work. This is a measurement-window artifact (and possibly GPU asynchronous execution, where a CUDA call can return before the GPU has finished the work) rather than a real zero-cost frame. Pinning down the exact mechanism with confidence would require adding explicit `torch.cuda.synchronize()` calls around the timed region — a worthwhile addition to `performance_analysis.py` for anyone extending this work, and noted here rather than silently smoothed over.

8. Summary of Findings

Metric	Finding	Recommendation
FPS	Wall-clock (real) FPS is 2.6-3.9× lower than inference-only FPS in both runs	Always report/trust wall-clock FPS for real-world claims
Inference time	YOLOv8s+fp16 is ~4× slower per frame than YOLOv8n+fp32	Model size is the dominant lever, precision mode is secondary
CPU usage	~1 core average, ~2 core peaks, out of 12 available	CPU is not the bottleneck; GPU inference dominates
Memory usage	Plateaus after ~1-3s, no growth over 300 frames	No evidence of a memory leak at this timescale
Tracking accuracy proxies	Larger model showed worse ID-stability proxy — likely detecting more small/hard objects, not necessarily 'worse' tracking	Needs ground-truth or manual video review to fully resolve
Pipeline overhead	~15-23ms/frame fixed cost outside the timed model call	Dominant bottleneck once a fast model is in use — target next